

Viva guide: DVC steps and command demos

GitHub: <https://github.com/TonmoyBishwas/laser-detector-mlsd>- DagsHub:
<https://dagshub.com/ttonmoy46/laser-detector-mlsd->

Local copies of the repo:

machine	folder	activate the venv
Windows PC (RTX 3090)	H:\Projects\laser-detector-mlsd	venv\Scripts\Activate.ps1
MacBook Air M4	~/Codes/laser-detector-mlsd-	source .venv/bin/activate

(venv) / (.venv) appears at the start of the prompt. On Windows, if PowerShell says "running scripts is disabled", run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once. Every `dvc` and `python` command below is the same on both machines. All outputs shown in this guide are real, copied from the Mac.

Part 1: how the project was set up

The models were trained with **semi-supervised learning** in an earlier project: a teacher YOLO26l trained on a few hand-labelled frames, pseudo-labelled the rest, and a student YOLO26n was trained on everything. For this course, that work is packaged into a DVC pipeline so data, models and results are versioned and reproducible.

1. **Copy the files into the repo** `data/raw/red/` , `data/raw/green/` (the final semi-supervised datasets), `models/released/red_yolo26n.pt` , `models/released/green_yolo26n.pt` (the student models).
2. `dvc init` Creates the `.dvc/` folder: the settings file (`config`) and the local cache (`.dvc/cache`).
3. `dvc add data/raw/red data/raw/green models/released/red_yolo26n.pt models/released/green_yolo26n.pt` For each item DVC:
 4. computes its fingerprint (MD5 hash),
 5. copies it into `.dvc/cache` ,

6. writes a small pointer file (e.g. `red_yolo26n.pt.dvc`) with the hash and size,
7. adds the real file to `.gitignore` automatically.
8. `dvc repro` Runs the pipeline in `dvc.yaml`: `prepare` → `train` → `evaluate`.
Outputs (`data/prepared/`, `models/model.pt`, `metrics/metrics.json`) are cached and their hashes are written to `dvc.lock`.
9. **Git commit and push to GitHub** `git add` the pointers, `dvc.yaml`, `dvc.lock`, `params.yaml` and the code, then `git commit` and `git push`. GitHub gets only code and small text files.
10. **Create a DagsHub repo** and connect it to the GitHub repo.
11. **Point DVC at DagsHub and log in** `dvc remote add -d origin https://dagshub.com/ttonmoy46/laser-detector-mlsd-.dvc` `dvc remote modify origin --local auth basic` `dvc remote modify origin --local user ttonmoy46` `dvc remote modify origin --local password <DagsHub token>`
The first line saves the address (committed). The other three are the login and go into `.dvc/config.local`, which is never committed.
12. `dvc push` Uploads every file the pointers (`.dvc` files and `dvc.lock`) refer to that DagsHub does not have yet: the datasets, the models and the pipeline outputs.
13. **Second machine: the MacBook (the reproducibility test)** `git clone` → `install` → `dvc pull` → `dvc repro` on a completely different computer (Apple M4, no NVIDIA GPU). Two things had to change, both now in Git:
14. `src/train.py` and `src/evaluate.py` had `device=0` (NVIDIA only). They now ask `src/device.py`, which picks NVIDIA GPU → Apple GPU (`mps`) → CPU.
`src/device.py` is listed as a dependency of train and evaluate in `dvc.yaml`.
15. The python.org Python on the Mac had no SSL certificates, so `dvc pull` failed with `CERTIFICATE_VERIFY_FAILED`. Fix: `export SSL_CERT_FILE=$(python -c "import certifi; print(certifi.where())")` (the Mac's `.venv/bin/activate` does this automatically).

After the change DVC saw the edited scripts and reran train and evaluate; the results matched the PC (section "Results on the Mac" in `VIVA_QA.md`).

1. One Git branch per laser + live detection

- `main` branch = red (dataset: red), green branch = green (dataset: green). Each branch has its own `params.yaml`, `dvc.lock` and metrics, so switching laser is `git checkout green + dvc checkout`.
- `main.py` runs the pipeline's model (`models/model.pt`) on the webcam / an RTSP stream, so the same command `python main.py` works for both lasers.

Git vs DVC in one line: Git (GitHub) stores code and small pointer files; DVC stores the big data and models in remote storage (DagsHub); `dvc pull` reads the pointers and downloads the real files.

How someone else reproduces it: `git clone` → `dvc pull` → `dvc repro`.

Part 2: what `dvc repro` actually does here

stage	does	input	output
prepare	checks every label, builds the clean dataset folder	<code>data/raw/red</code> (1,748 images)	<code>data/prepared/</code>
train	does not train by default: copies the semi-supervised model (<code>train.mode: released</code>)	<code>models/released/red_yolo26n.pt</code>	<code>models/model.pt</code>
evaluate	tests the model on the 216 hand-labelled test frames	<code>models/model.pt</code> , <code>data/prepared</code>	<code>metrics/metrics.json</code>

`train.mode: train` would train the student step again (YOLO26n, 1280 px, 100 epochs, batch 16, seed 0, same augmentation). It does not redo the teacher or the pseudo-labels: their result is the dataset in `data/raw`.

If asked "why does your training stage not train?": "Training takes several hours on an RTX 3090, so by default the stage uses our trained semi-supervised model. Setting `train.mode: train` in `params.yaml` retrains it with the same settings; DVC

sees the parameter changed and reruns training and evaluation."

Part 3: command demos

Every demo below is safe; each one says how to undo it.

`dvc status` : "is anything out of date?"

```
$ dvc status
Data and pipelines are up to date.
```

Say: "DVC compares the current hashes of all data, code and parameters with the ones saved in `dvc.lock`. Nothing changed, so nothing needs to rerun."

`dvc dag` : "show your pipeline"

```
$ dvc dag
+-----+
| data/raw/red.dvc |
+-----+
      *
      *
      *
+-----+ +-----+
| prepare |***| | models/released/red_yolo26n.pt.dvc |
+-----+ +-----+
      *          *****          *
      *          *****          *
      *          *****          *
      *          *****          *
      *          +-----+
      *          | train |
      *          *+-----+
      *          **          **
      *          ***          ***
      *          **          **
      *          +-----+
      *          | evaluate |
      *          +-----+
+-----+
| models/released/green_yolo26n.pt.dvc |
+-----+
+-----+
| data/raw/green.dvc |
+-----+
```

Say: "DAG = directed acyclic graph. It shows which stage depends on which, so DVC knows the order and what to rerun when something changes." The green dataset and green model show as separate boxes because this branch uses red.

A parameter changes

In `params.yaml`, under `evaluate`, change `conf: 0.25` to `conf: 0.5`, then:

```
$ dvc status
evaluate:
  changed deps:
    params.yaml:
      modified:          evaluate

$ dvc params diff
Path      Param      HEAD      workspace
params.yaml  evaluate.conf  0.25      0.5

$ dvc repro
'data/raw/red.dvc' didn't change, skipping
Stage 'prepare' didn't change, skipping
'models/released/red_yolo26n.pt.dvc' didn't change, skipping
Stage 'train' didn't change, skipping
Running stage 'evaluate':
> python src/evaluate.py
              all          216          113          0.974          0.991          0.994          0.81
3
              all          48           22          0.996          0.955          0.955          0.62
5
Updating lock file 'dvc.lock'

$ dvc metrics diff
Path      Metric      HEAD      workspace      Change
metrics/metrics.json  test_image_level.accuracy  0.9815  0.9769      -0.0046
metrics/metrics.json  test_image_level.misses    3        4           1
```

(The Ultralytics progress bars are left out of the `dvc repro` output.) **Say:** "Only `evaluate` uses `evaluate.conf`, so only that stage reruns. The box metrics don't change, because mAP is computed over all thresholds; only the image-level laser / no-laser decision uses `conf`, and at 0.5 one more weak dot is missed." **Undo:** set it back to `0.25`, then `dvc repro`:

```

$ dvc repro
'data/raw/red.dvc' didn't change, skipping
Stage 'prepare' didn't change, skipping
'models/released/red_yolo26n.pt.dvc' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' is cached - skipping run, checking out outputs
Updating lock file 'dvc.lock'

$ dvc status
Data and pipelines are up to date.

```

Say: "DVC's run cache remembers the result for `conf: 0.25`, so it restores it instead of evaluating again."

The dataset changes: switch branches (red ↔ green)

Each laser lives on its own Git branch. Git switches the small files; DVC then switches the big ones to match:

```

$ git checkout green
Switched to branch 'green'

$ dvc status
prepare:
  changed outs:
    modified:      data/prepared
train:
  changed deps:
    modified:      data/prepared
  changed outs:
    modified:      models/model.pt
evaluate:
  changed deps:
    modified:      data/prepared
    modified:      models/model.pt

$ dvc checkout
M      data/prepared/
M      models/model.pt

$ dvc status
Data and pipelines are up to date.

```

Say: "After `git checkout`, `dvc.lock` describes the green run but the files on disk are still red, so `dvc status` says they are modified. `dvc checkout` copies the green versions out of the cache in a second, no download and no rerun."

Compare the two branches without switching:

```

$ dvc params diff main green
Path      Param    main    green
params.yaml dataset  red     green

$ dvc metrics diff main green
Path      Metric          main    green    Change
metrics/metrics.json test.mAP50      0.9938  0.7      -0.2938
metrics/metrics.json test.mAP50_95   0.813   0.2861   -0.5269
metrics/metrics.json test.precision   0.9739  0.8333   -0.1406
metrics/metrics.json test.recall      0.9905  0.8329   -0.1576
metrics/metrics.json test_image_level.accuracy 0.9815  0.9917   0.0102
metrics/metrics.json test_image_level.false_alarms 1       0        -1
metrics/metrics.json test_image_level.images 216     120      -96
metrics/metrics.json test_image_level.misses 3        1        -2
metrics/data_stats.json test.images     216     120      -96
metrics/data_stats.json train.images       1484    820      -664

```

(shortened; the full output also lists the valid metrics and laser / no-laser counts.)

Back to red: `git checkout main` then `dvc checkout`.

The same thing without branches: change `dataset: red` to `dataset: green` in `params.yaml`; `dvc status` shows all three stages out of date and `dvc repro` reruns them. **Say:** "Every stage depends on the dataset, so all of them rerun." **Undo:** set it back to `red`, then `dvc repro`: the stages say `cached – skipping run`, because DVC remembers earlier results.

`dvc add` : "how did you track the data?"

```

dvc add data/raw/red

```

Nothing changed, so the pointer stays the same. Open `data/raw/red.dvc` to show the hash (`md5`), size and number of files. **Say:** " `dvc add` saves the file's hash in a small `.dvc` pointer, copies the file into the cache, and adds it to `.gitignore`. Git stores the pointer; DVC stores the file."

dvc pull : "get the data back"

```

$ rm models/released/red_yolo26n.pt # Windows: Remove-Item models\released\red_yolo26n.pt
$ dvc status
train:
  changed deps:
    deleted:      models/released/red_yolo26n.pt
models/released/red_yolo26n.pt.dvc:
  changed outs:
    deleted:      models/released/red_yolo26n.pt

$ dvc pull
A      models/released/red_yolo26n.pt
1 file added

$ dvc status
Data and pipelines are up to date.
```

Say: "dvc pull reads the pointer files and downloads the real files from DagsHub. Another person would run `git clone`, then `dvc pull`, then `dvc repro`."

dvc push : "upload your data"

```

$ dvc push
Everything is up to date.
```

Say: "Everything is already on DagsHub. If I added new data, `dvc push` would upload only the new or changed files." (When the green branch was created, `dvc push` uploaded its new pipeline outputs.)

dvc init : explain, don't run

It has already been run here; running it again only says "already initialized". **Say:** "`dvc init` creates the `.dvc` folder with the settings and the cache. It is the first command in a DVC project, like `git init`."

Live demo: the model on a webcam / RTSP stream (`main.py`)

```

python main.py # webcam (detect.source in params.yaml)
python main.py --source "rtsp://user:pass@192.168.1.20:554/stream1"
python main.py --source <image or video file>
```

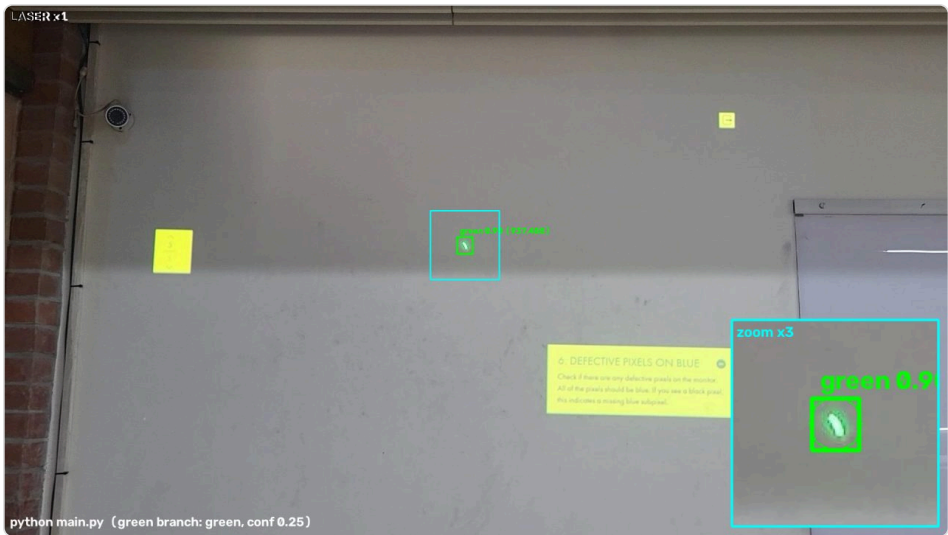
`main.py` loads `models/model.pt` (the output of the train stage) and the `detect:` settings from `params.yaml`. A window shows the frame with the detected dot; `q` quits, `s` saves a snapshot.

```
$ python main.py --source data/raw/red/test/images/20260517_162427_f00480.jpg
model: models/model.pt (red) | conf: 0.8 | device: mps | imgsz: 1280 | source: data/raw/red/test/images/20260517_162427_f00480.jpg
red conf=0.94 box=(1730,184,1752,206)
1 laser(s) found
```

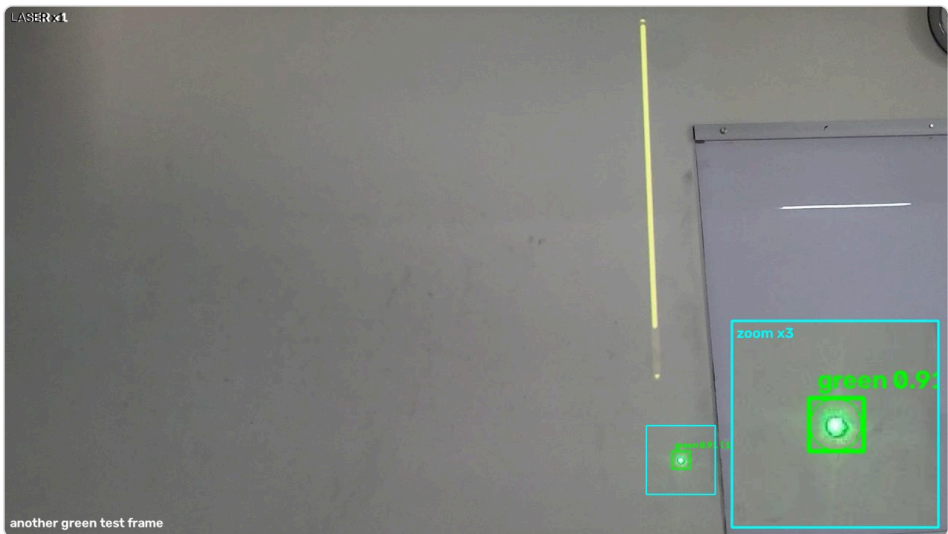


Red laser found by main.py on the main branch

On the green branch the same command uses the green model:



Green laser found by main.py on the green branch



A bright fluorescent tube is ignored; only the dot is found



A frame without a laser: nothing detected

`main.py` **refuses to run the wrong model.** It checks that `params.yaml`, `dvc.lock` and the real `models/model.pt` (by its MD5 hash) all agree:

```
$ git checkout main
$ python main.py      # forgot dvc checkout
models/model.pt is not the red model that dvc.lock records.
Run `dvc checkout` (after switching branches) or `dvc repro`.

$ python main.py      # dataset: green in params.yaml, no dvc repro yet
params.yaml says dataset: green, but dvc.lock has the model for red.
Run `dvc repro` to rebuild models/model.pt.
```

Say: "This is the same hash idea DVC uses: the lock file says which model belongs to this version, and the program checks it before using it."

`detect.py` is the hands-on version: `python detect.py --laser red|green|both` runs a released model directly, without the pipeline (useful to compare both lasers at once).

Part 4: likely viva questions, short answers

- **What does `dvc repro` do?** Runs the pipeline in `dvc.yaml`, rerunning only the stages whose inputs (data, code, parameters) changed since `dvc.lock` was written.
- **Why DVC?** Git can't handle big data and models well; DVC versions them, stores them in remote storage, and makes the pipeline reproducible.
- **What is `dvc.yaml`?** The pipeline definition: each stage's command, dependencies, parameters and outputs.
- **What is `params.yaml`?** All settings in one file; each stage declares which ones it uses.
- **What is `dvc.lock`?** The hashes of every input and output from the last run; DVC compares against it to decide what to rerun.
- **How does DVC know which stage to rerun?** It compares current hashes with `dvc.lock`; a changed dependency or parameter makes that stage and everything after it out of date.
- **What happens if the dataset changes?** Its hash changes, so prepare, train and evaluate all rerun.
- **What happens if a parameter changes?** Only the stages that use that parameter (and stages after them) rerun.
- **Git vs DVC?** Git stores code and pointers; DVC stores the data and models (DagsHub).
- **`dvc push` / `dvc pull`?** Upload / download the real data and models to / from remote storage.
- **How can someone reproduce your experiment?** `git clone` → `dvc pull` → `dvc repro`. We did exactly that on a MacBook and got the same results.
- **Why two branches?** One per laser. `git checkout` switches code, params and `dvc.lock`; `dvc checkout` switches the data and model to match.
- **Why doesn't changing `detect` in `params.yaml` make anything out of date?** No stage lists the `detect` parameters in `dvc.yaml`; only `main.py` reads them.

Note

The pipeline picks the device itself (`src/device.py`): NVIDIA GPU on the PC, Apple GPU (`mps`) on the Mac, CPU elsewhere. So every demo, including `dvc repro`, works on both machines. On the Mac the whole pipeline takes about 30 seconds with

```
train.mode: released.
```